개발자 작업실 꾸미기 최종 보고서

2026-11-04 · 경남캠퍼스 @민경

1. 한눈에 보기

제 Windows 컴퓨터에 개발자가 일하는 '작업실'을 직접 차렸고, 모든 과정을 실제 실행 결과로 확인해 GitHub에 기록했습니다.

작업실에 필요한 도구는 세 가지였습니다.

도구	하는 일	쉬운 비유
터미널	글자로 컴퓨터에게 명령하기	마우스 대신 말로 시키는 비서
Docker	프로그램을 상자에 담아 어디서나 똑같이 실행하기	틀 하나로 똑같은 붕어빵을 몇 개든 찍어 내는 붕어빵 기계
Git / GitHub	작업 기록을 저장하고 인터넷에 공유하기	내 일기장 / 온라인 공유 앨범

결과물 저장소: github.com/coolgirl2046/workstation

📖 읽기 전에: 처음 보는 낱말 풀이

보고서 곳곳에 나오는 낱말을 먼저 쉽게 풀어 둡니다. 본문 중간의 💬 **쉬운 풀이** 상자도 같은 목적의 주석입니다.

낱말	쉬운 뜻
명령어	컴퓨터에게 글자로 내리는 심부름 쪽지입니다
리눅스 / Ubuntu(우분투)	Windows처럼 컴퓨터를 움직이는 기본 프로그램(운영체제)입니다. Ubuntu는 리눅스 종류 중 하나입니다
폴더(디렉터리)	파일을 담는 서랍입니다
서버 / 웹 서버	손님(브라우저)이 부르면 웹 페이지를 내어 주는 프로그램입니다. 식당 주방과 비슷합니다
NGINX(엔진엑스)	많이 쓰이는 웹 서버 프로그램의 이름입니다
브라우저	크롬, 엣지처럼 인터넷 화면을 보는 프로그램입니다
localhost	"바로 이 컴퓨터"라는 뜻의 주소입니다
포트	한 컴퓨터에 달린 여러 개의 출입문 번호입니다

낱말	쉬운 뜻
이미지	그림이 아니라, 프로그램을 찍어 내는 붕어빵 틀입니다
컨테이너	그 틀로 찍어 내, 실제로 돌아가고 있는 붕어빵입니다
빌드	설계도(Dockerfile)대로 붕어빵 틀(이미지)을 만드는 일입니다
로그	프로그램이 스스로 적는 일기(기록)입니다
저장소	프로젝트 파일과 변경 기록을 모아 둔 보관함입니다
커밋	"여기까지 저장" 하고 메모를 붙여 남기는 기록 한 장입니다
push(푸시)	내 컴퓨터의 기록을 인터넷(GitHub)으로 올려 보내는 일입니다
README	저장소의 첫 화면에 보이는 설명서입니다

2. 내 작업실 환경

저는 경남캠퍼스 교육생으로, 개인 Windows PC에서 과제를 수행했습니다. 미션에 나오는 OrbStack은 '서울 추가 내용'으로 붙은 안내입니다. 서울캠퍼스에서는 보안 정책 때문에 sudo(관리자 권한) 사용이 제한될 수 있어서, 그 문제를 피하려고 OrbStack을 씁니다. 저에게는 해당하지 않아, 공통 요구사항인 'Docker 설치 및 기본 점검'에 따라 WSL2 + Docker Desktop을 직접 설치했습니다. `docker` 명령어 사용법은 똑같습니다.

항목	내 환경	쉽게 말하면
운영체제	Windows 10 + WSL2 Ubuntu	Windows 안에 리눅스 컴퓨터를 하나 더 놓은 것
셸 / 터미널	bash / Ubuntu 터미널, VSCode 터미널	명령을 입력하는 창
Docker	Docker Desktop 4.97.0 (엔진 29.9.2)	붕어빵 틀(이미지)을 만들고, 그 틀로 붕어빵(컨테이너)을 찍어 내는 프로그램
Git	2.53.0	작업 기록 프로그램
편집기	VSCode + WSL 확장	코드를 쓰는 메모장

WSL2는 'Windows용 리눅스 하위 시스템'이라는 뜻입니다. Windows 컴퓨터 안에서 개발자들이 많이 쓰는 리눅스를 앱처럼 띄워 쓸 수 있게 해줍니다.

> 💬 **쉬운 풀이**
>
> - **셸(bash)**: 제가 적은 명령어를 컴퓨터가 알아듣게 옮겨 주는 통역사입니다. 터미널은 그 통역사와 대화하는 창입니다.
> - **엔진**: Docker 안에서 실제로 상자를 만들고 돌리는 심장 부분입니다.
> - **확장**: 앱에 끼워 쓰는 추가 기능입니다. WSL 확장 덕분에 VSCode가 Windows 안의 리눅스 파일을 열 수 있습니다.

3. 과제 목표 6가지, 쉽게 설명하기

미션에서 "스스로 설명할 수 있어야 한다"고 한 6가지를, 한 줄 답 → 비유 → 제가 직접 해본 것 순서로 정리했습니다.

① 절대 경로와 상대 경로

- **한 줄 답**: 절대 경로는 맨 꼭대기(`/`)부터 적는 전체 주소이고, 상대 경로는 지금 제가 있는 곳을 기준으로 적는 주소입니다.
- **비유**: 휴대폰 사진첩을 '사진첩 > 블로그 > 맛집' 순서로 정리했다고 해 보겠습니다. 절대 경로는 "사진첩 첫 화면 → 블로그 → 맛집"처럼 처음부터 길을 다 말하는 방식이라, 지금 어느 앨범을 보고 있든 같은 곳에 도착합니다. 상대 경로는 '블로그' 앨범을 보고 있을 때 "여기서 맛집 앨범"이라고만 말하는 방식이라, 다른 앨범에서 같은 말을 하면 길을 잃습니다.
- **제가 해본 것**: `cd /home/pc/practice/docs` (절대 경로)로 들어간 뒤 `cd ../..` (상대 경로, 두 단계 위)로 나왔고, `pwd`로 위치가 `/home/pc`가 된 것을 확인했습니다. `..`은 한 단계 위, `~`는 제 홈 폴더를 뜻합니다.

제 Dockerfile 속 예시: `COPY app/ /usr/share/nginx/html/` 한 줄에 둘이 모두 들어 있습니다. 앞의 `app/`는 "지금 있는 workstation 폴더 안의 app"이라는 상대 경로이고, 뒤의 `/usr/share/nginx/html/`는 맨 꼭대기 `/`부터 적은 절대 경로입니다. 구별법은 하나입니다. 맨 앞이 `/`로 시작하면 절대 경로, 아니면 상대 경로입니다.

> 💬 **쉬운 풀이**: `cd`는 "이 서랍으로 이동해"(change directory), `pwd`는 "나 지금 어디 있어?"(print working directory)라는 명령입니다. `/`는 모든 서랍을 품은 맨 꼭대기 서랍입니다.

② 파일 권한(r/w/x)과 755, 644

- **한 줄 답**: 파일마다 "누가 읽고(r), 고치고(w), 실행하거나 들어갈(x) 수 있는지"가 정해져 있고, 이를 숫자 세 자리로 적습니다.
- **숫자 규칙**: 읽기=4, 쓰기=2, 실행=1을 더합니다. 세 자리는 순서대로 **나(주인) / 우리 팀(그룹) / 그 외 모두**입니다.
  - 7 = 4+2+1 → 읽기·쓰기·실행 모두

    - 6 = 4+2 → 읽기·쓰기
    - 5 = 4+1 → 읽기·실행
    - 4 → 읽기만
- **비유**: 일기장에 붙인 안내문입니다. `644`는 "나는 읽고 쓸 수 있고, 가족과 손님은 읽기만", `755`는 "나는 다 할 수 있고, 나머지는 열어서 볼 수만 있음"입니다.
- **제가 해본 것**: 파일 `hello.txt`를 644 → 600(나만 읽고 쓰기) → 644로, 폴더 `docs`를 755 → 700(나만) → 755로 바꿨습니다. 실수로 `chmod 70`을 입력했더니 주인인 저조차 폴더에 들어갈 수 없었습니다(`Permission denied`). 폴더의 x는 "안에 들어갈 수 있는 권한"이라는 것을 이때 몸으로 알게 됐습니다.

> 💬 **쉬운 풀이**: `chmod`는 "권한 안내문을 바꿔"(change mode)라는 명령입니다. `Permission denied`는 "허락되지 않았습니다"라는 뜻의 영어 문장입니다. 그룹은 같은 팀으로 묶인 사람들입니다.

③ 기존 이미지로 "커스텀 이미지" 만들기

- **한 줄 답**: 다른 사람이 만들어 둔 이미지를 바탕으로, 제가 필요한 것만 얹어 나만의 이미지를 만드는 것입니다. 그 방법을 적은 파일이 Dockerfile입니다.
- **비유**: 가게에서 파는 기본 붕어빵 틀(기존 이미지)을 사 와서, 제 무늬를 새겨 넣어 나만의 틀로 고치는 것입니다. Dockerfile은 "어떤 틀을 골라 어디를 고칠지" 적은 설계도입니다.
- **제가 해본 것**: 웹 서버 프로그램 NGINX(엔진엑스)가 들어 있는 `nginx:alpine`을 바탕으로 4가지를 바꿨습니다.

내가 추가한 것	쉬운 뜻
`LABEL`	이미지에 이름표 붙이기
`ENV APP_ENV=dev`	"지금은 개발용"이라는 설정값 적어두기
`COPY app/ ...`	기본 환영 페이지를 내가 만든 페이지로 교체
`HEALTHCHECK`	30초마다 "서버 살아 있니?" 스스로 점검

빌드 후 `docker ps`에 `(healthy)`가 뜨고, `docker inspect`에 제가 붙인 이름표와 설정값이 보여 적용된 것을 확인했습니다.

증거 기록: 제가 쓴 레시피(Dockerfile)

```
FROM nginx:alpine
LABEL org.opencontainers.image.title="my-workstation-web"
ENV APP_ENV=dev
COPY app/ /usr/share/nginx/html/
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -q -O /dev/null http://localhost/ || exit 1
EXPOSE 80
```

> 💬 **쉬운 풀이: 레시피 한 줄씩 읽기**
>
> - `FROM nginx:alpine` : 기본 틀 고르기. 웹 서버 NGINX가 든 틀을 바탕으로 씁니다. `alpine` 은 크기가 아주 작은 리눅스 종류라는 뜻입니다.
> - `LABEL ...` : 상자에 "my-workstation-web"이라는 이름표를 붙입니다.
> - `ENV APP_ENV=dev` : 상자 안에 "지금은 개발(development) 중"이라는 메모를 붙입니다.
> - `COPY app/ /usr/share/nginx/html/` : 제 PC의 `app` 서랍 속 페이지를, 상자 안에서 손님에게 보여 주는 진열대(`html` 폴더)로 옮겨 담습니다.
> - `HEALTHCHECK ...` : 30초마다 스스로 문을 두드려 보고, 3초 안에 대답이 없으면 "아프다"고 표시합니다. `wget` 은 페이지를 불러오는 작은 도구입니다.
> - `EXPOSE 80` : "이 상자는 안쪽 80번 문을 씁니다"라는 안내판입니다. 실제로 바깥과 문을 잇는 일은 실행할 때 `-p` 가 합니다.

첫 줄 `FROM nginx:alpine` 이 "기존 이미지를 바탕으로 쓴다"는 뜻이고, 그 아래 줄들이 제가 새겨 넣은 무늬입니다.

증거 기록: 빌드(레시피대로 이미지 만들기) 성공

```
$ docker build -t my-web:1.0 .
[+] Building 109.8s (7/7) FINISHED
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:785835ea65db...   65.9s
 => [2/2] COPY app/ /usr/share/nginx/html/                              1.1s
 => => naming to docker.io/library/my-web:1.0                           1.4s

$ docker images
IMAGE                ID             DISK USAGE   CONTENT SIZE
my-web:1.0           5068b6c124ad       93.6MB         26.3MB
```

> 💬 **쉬운 풀이**
>
> - `-t my-web:1.0` : 상자에 붙이는 이름표입니다. `t` 는 tag(꼬리표)의 첫 글자이고, `my-web` 은 이름, `:1.0` 은 버전(몇 번째 판인지)입니다.
> - 맨 끝 `.` : "지금 있는 이 서랍의 레시피를 써라"라는 뜻입니다.
> - `sha256:785835...` : 상자마다 다른 지문 같은 고유번호입니다.
> - `109.8s` : 포장에 걸린 시간(약 110초)입니다. `93.6MB` 는 상자의 크기입니다.

`FINISHED` 는 빌드가 끝까지 성공했다는 뜻이고, `docker images` 목록에 제 이미지 `my-web:1.0` 이 생겼습니다.

증거 기록: 제가 붙인 이름표와 설정값이 들어갔는지 확인

```
$ docker inspect my-web-8080 --format '{{.Config.Labels}} {{.Config.Env}}'
map[... org.opencontainers.image.title:my-workstation-web] [... APP_ENV=dev]

$ docker ps
IMAGE        STATUS                    PORTS                     NAMES
my-web:1.0   Up 46 minutes (healthy)   0.0.0.0:8081->80/tcp      my-web-8081
my-web:1.0   Up 46 minutes (healthy)   0.0.0.0:8080->80/tcp      my-web-8080
```

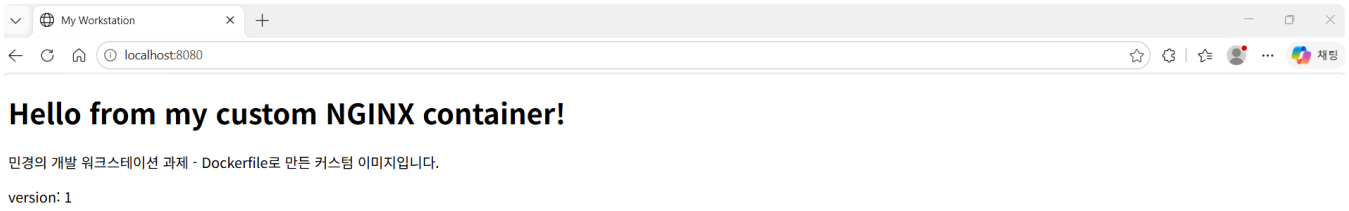
> 💬 **쉬운 풀이**: `docker inspect` 는 "이 상자 속을 자세히 보여줘", `docker ps` 는 "지금 켜져 있는 상자 목록을 보여줘"라는 명령입니다. `Up 46 minutes` 는 46분째 켜져 있다는 뜻이고, `0.0.0.0:8080->80/tcp` 는 "바깥 8080번 문으로 오면 안쪽 80번 문으로 안내"라는 연결 표시입니다.

`my-workstation-web` (LABEL)과 `APP_ENV=dev` (ENV)가 보이고, `(healthy)` 는 HEALTHCHECK 자가 점검을 통과했다는 뜻입니다. 위 기록은 README 10·11장의 실제 실행 결과에서 필요한 줄만 옮긴 것입니다(`...` 은 생략 표시).

④ 포트 매핑이 필요한 이유

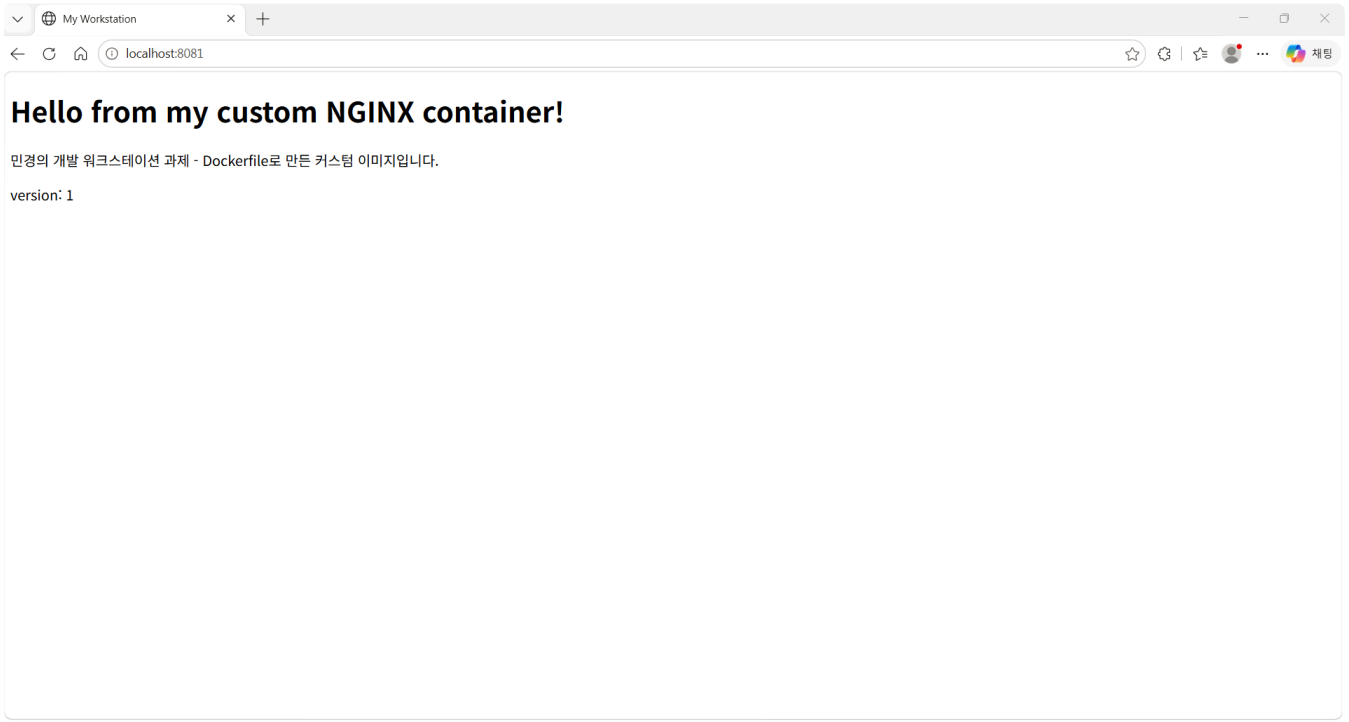
- **한 줄 답**: 컨테이너는 바깥과 분리된 상자 안에 있어서, 바깥에서 들어갈 문을 따로 이어줘야 접속할 수 있습니다. 그 연결이 포트 매핑입니다.
- **비유**: 건물의 바깥문과 안쪽 방문입니다. 손님은 바깥문(내 컴퓨터의 8080번)으로만 들어올 수 있고, 안내원이 컨테이너 안쪽 방문(80번)까지 데려다줍니다. `-p 8080:80` 은 "8080번 바깥문으로 온 손님은 80번 방문으로"라는 설정입니다.
- **왜 좋은가**: 두 컨테이너가 둘 다 안에서 80번을 써도, 바깥 번호만 다르게 주면 동시에 띄울 수 있습니다.
- **제가 해본 것**: 같은 이미지를 8080, 8081 두 번호로 실행하고, 브라우저 주소창에 `localhost:8080` , `localhost:8081` 을 넣어 제 페이지가 뜨는 것을 캡처했습니다.

증거 화면: 8080번으로 접속



주소창의 `localhost:8080`은 "내 컴퓨터의 8080번 문"이라는 뜻입니다. 이 문을 두드리자 컨테이너 안의 웹 서버가 제가 만든 페이지("Hello from my custom NGINX container!")를 보여주었습니다. 화면의 `version: 1`은 바인드 마운트 실습으로 파일을 고치기 전 상태라는 표시입니다.

증거 화면: 8081번으로 접속



같은 이미지로 만든 두 번째 컨테이너입니다. 문 번호만 8081로 다르게 주었는데도 같은 페이지가 뜨는 것을 보면, "안에서는 둘 다 80번, 밖에서는 다른 번호"라는 포트 매핑의 원리를 확인할 수 있습니다.

⑤ Docker 볼륨(오래 남는 데이터)

- **한 줄 답**: 컨테이너를 지우면 그 안의 데이터도 같이 사라집니다. 볼륨은 컨테이너와 따로 보관되는 저장 공간이라, 컨테이너를 지워도 남습니다. 이렇게 오래 남는 성질을 '영속성', 그런 데이터를 '영속 데이터'라고 부릅니다.
- **비유**: 컨테이너는 호텔 방, 볼륨은 호텔 개인 사물함입니다. 체크아웃(삭제)하면 방은 치워지지만, 사물함 속 짐은 그대로라 다음 방에서 다시 꺼낼 수 있습니다.
- **제가 해본 것**: 볼륨 `mydata`를 만들어 컨테이너 `voltest1`에 연결하고 `hello-volume`이라는 글을 저장했습니다. `voltest1`을 삭제한 뒤 새 컨테이너 `voltest2`에 같은 볼륨을 연결하니 `hello-volume`이 그대로 있었습니다.

> 💬 **쉬운 풀이**: `mydata`는 사물함에 붙인 이름표이고, `voltest1`, `voltest2`는 제가 지어 준 컨테이너(호텔 방) 이름입니다. 방 번호가 바뀌어도 같은 사물함 이름만 알려 주면 짐을 다시 꺼낼 수 있습니다.

⑥ Git과 GitHub의 차이

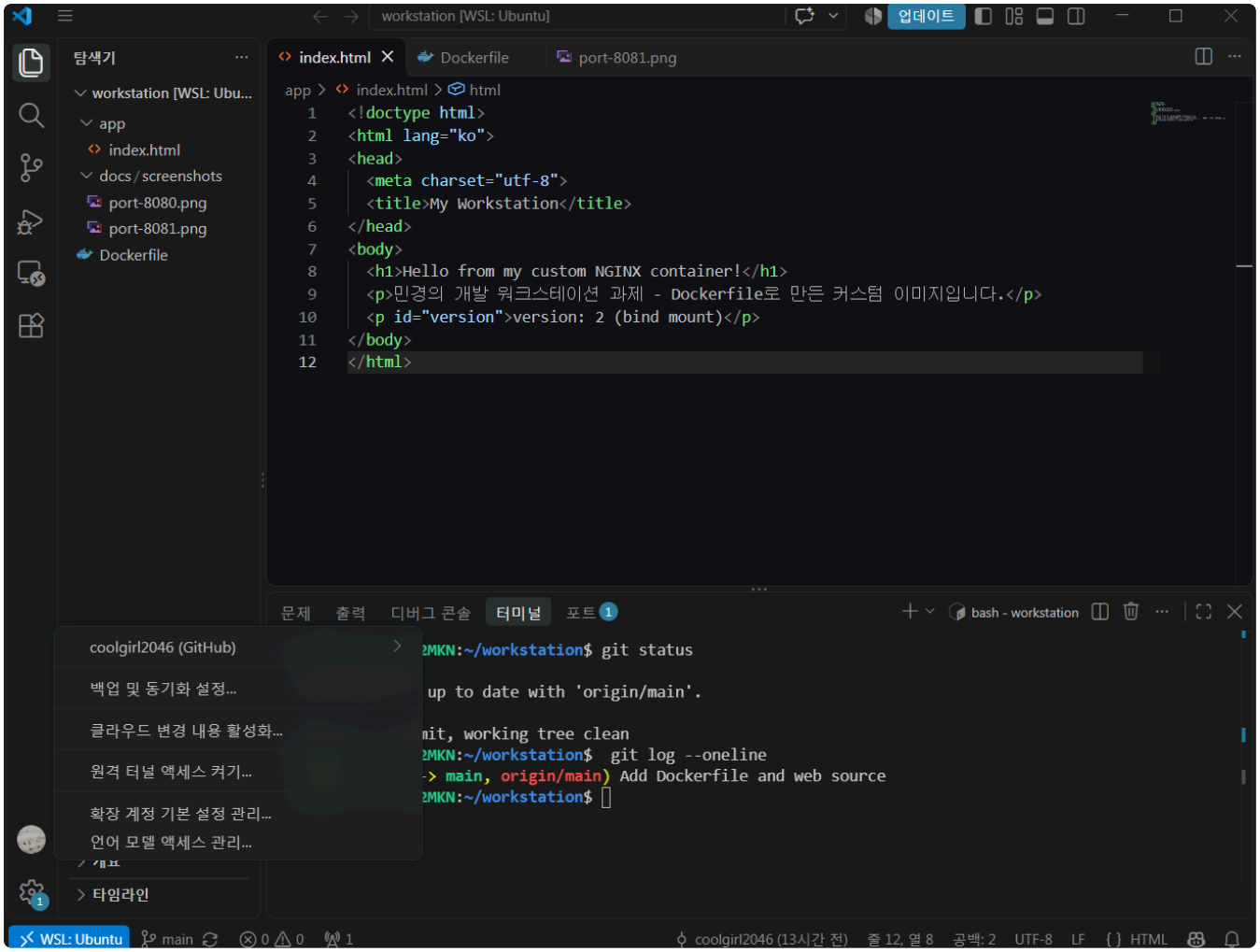
- **한 줄 답**: Git은 제 컴퓨터에서 변경 기록을 남기는 도구(로컬 버전 관리)이고, GitHub는 그 기록을 인터넷에 올려 여러 사람이 함께 보고 고치는 웹사이트(원격 협업 플랫폼)입니다. '로컬'은 내 컴퓨터 안, '원격'은 인터넷 저편의 다른 컴퓨터라는 뜻입니다.
- **비유**: Git은 내 책상 위 일기장, GitHub는 가족이 함께 보는 온라인 공유 앨범입니다. 일기는 인터넷 없이도 쓸 수 있고, 공유하고 싶을 때만 앨범에 올립니다.
- **제가 해본 것**:

어디서	명령	쉬운 뜻
Git(내 PC)	`git add .`	저장할 파일 바구니에 담기
Git(내 PC)	`git commit -m "메모"`	메모와 함께 저장 기록 남기기
GitHub(인터넷)	`git push`	인터넷 저장소로 올리기

커밋 기록은 `1b3da91` → `757328a` → `d9c2722`처럼 고유 번호로 쌓였고, GitHub 화면에서도 같은 번호를 확인했습니다.

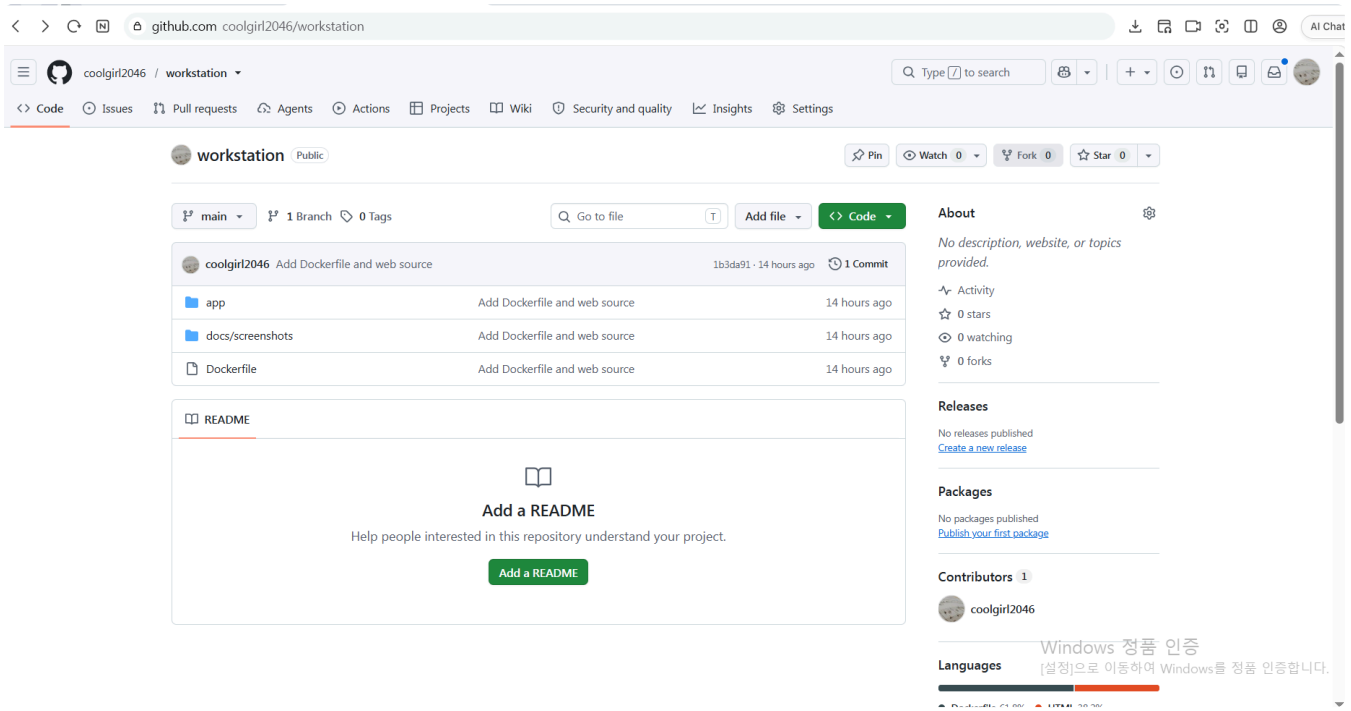
> 💬 **쉬운 풀이**: `1b3da91` 같은 번호는 앨범 사진 뒷면에 적는 고유번호처럼 기록마다 하나씩 붙습니다. `-m`은 message(메모), `origin`은 GitHub 쪽 저장소의 별명, `main`은 기록이 쌓이는 기본 줄기 이름입니다.

증거 화면: VSCode의 GitHub 로그인



VSCode 왼쪽 아래 계정 메뉴에 `coolgirl2046 (GitHub)`이 보입니다. VSCode가 제 GitHub 계정에 로그인되어 있다는 뜻입니다. 아래 터미널의 `up to date with 'origin/main'`은 "내 PC의 기록과 GitHub의 기록이 같다"는 뜻이고, 왼쪽 아래 `WSL: Ubuntu`는 VSCode가 Windows 안의 리눅스에 연결돼 있다는 표시입니다.

증거 화면: GitHub에 올라간 저장소



주소창의 `github.com/coolgirl2046/workstation`이 제 저장소의 인터넷 주소입니다. `Public`은 누구나 볼 수 있다는 뜻이고, 제 PC에서 만든 `app`, `docs/screenshots`, `Dockerfile`이 그대로 올라가 있습니다. 이 화면은 README를 올리기 전에 찍었습니다.

4. 함께 이해한 개념

과제 목표를 설명하려면 아래 네 가지를 먼저 알아야 했습니다.

개념	쉬운 비유	제가 확인한 것
이미지 vs 컨테이너	이미지는 붕어빵 틀, 컨테이너는 그 틀로 구운 붕어빵	이미지 `my-web:1.0` 하나로 컨테이너 2개(8080, 8081)를 만들었습니다
attach vs exec	라이브 방송으로 치면 attach는 진행자 자리에 들어가기(내가 나가면 방송 끝), exec는 게스트로 들어가기(내가 나가도 방송 계속)	exec로 들어갔다 나오면 컨테이너가 계속 켜져 있었고(Up), attach로 들어갔다 `exit` 하면 꺼졌습니다(Exited)
바인드 마운트	블로그 글을 캡처(복사본)가 아니라 링크로 보내기. 원본을 고치면 바로 바뀜	제 PC의 `index.html`을 고치자, 연결된 8082는 바로 `version: 2`로 바뀌고 복사본을 가진 8080은 `version: 1` 그대로였습니다
Docker Compose (보너스)	매번 말로 주문하던 것을 레시피 카드 한 장에 적어두기	긴 `docker run` 명령을 `docker-compose.yml`에 적고 `docker compose up` 한 줄로 실행했습니다

> 💬 **쉬운 풀이**: attach는 "달라붙다", exec는 execute(실행하다)의 줄임말입니다. `exit`는 "나가기", `Exited`는 "꺼진 상태", `Up`은 "켜진 상태"라는 뜻입니다. 바인드 마운트의 마운트(mount)는 "연결해 붙이기"라는 뜻입니다.

바인드 마운트와 볼륨의 차이는 "내가 아는 폴더를 연결하느냐(바인드 마운트), Docker가 따로 관리하는 사물함을 연결하느냐(볼륨)"로 정리했습니다.

5. 왜 이렇게 설계할까

미션은 "따라 치기"가 아니라 "왜 이런 구조가 필요한지" 설명하는 것을 요구합니다. 실습에서 확인한 결과로 네 가지 이유를 정리했습니다.

설계 원칙	왜 필요한가	실습에서 확인한 것
이미지와 컨테이너의 분리	틀(이미지)을 한 번 만들면 똑같은 결과물(컨테이너)을 몇 개든 찍어낼 수 있습니다	이미지 `my-web:1.0` 하나로 8080·8081 컨테이너 2개를 똑같이 실행했습니다
격리된 실행 환경	상자끼리 서로 섞이지 않아서, 하나를 고치거나 지워도 다른 것에 영향이 없습니다	8080·8081 두 컨테이너가 안에서 똑같이 80번 문을 쓰는데도 서로 부딪치지 않고 동시에 실행됐습니다
포트 연결	격리된 상자에 바깥에서 들어갈 문을 필요한 만큼만 엽니다	`-p 8080:80`, `-p 8081:80`으로 같은 80번을 두 번호로 열었습니다
저장소 연결	상자는 지우고 다시 만드는 물건이라, 남겨야 할 데이터는 상자 밖에 둡니다	컨테이너를 지워도 볼륨 `mydata`의 `hello-volume`은 남았습니다

이 네 가지가 합쳐져 "내 컴퓨터에서만 돼요" 문제를 줄입니다. 레시피(Dockerfile, `docker-compose.yml`)만 있으면 다른 사람 컴퓨터에서도 같은 명령으로 같은 결과를 다시 만들 수 있기 때문입니다. 이것이 미션이 말하는 "재현 가능한 환경"입니다.

6. 수행 과정 요약

모든 단계를 터미널 명령으로 수행했고, 명령과 결과는 README에 그대로 기록했습니다.

1. **작업실 준비**: WSL2와 Ubuntu 설치, Docker Desktop 설치 후 `docker --version`, `docker info`로 작동 확인
2. **터미널 연습**: 폴더 만들기, 파일 복사·이동·이름 변경·삭제, 숨김 파일 보기
3. **권한 실습**: 파일 1개·폴더 1개 권한을 바꾼 뒤 원래대로 복구
4. **Docker 기본**: `hello-world` 실행, ubuntu 컨테이너 안에서 `ls`·`echo`, 로그·사용량 확인
5. **내 이미지 만들기**: Dockerfile 작성 → 빌드 → 8080·8081 두 포트로 실행 → 브라우저 접속 캡처

6. **데이터 연결**: 바인드 마운트로 수정 즉시 반영 확인, 볼륨으로 컨테이너 삭제 후에도 데이터 유지 확인
7. **Git/GitHub**: 사용자 설정, 커밋, GitHub에 올리기, VSCode GitHub 로그인 캡처
8. **보너스**: Docker Compose로 같은 웹 서버를 실행(`up`)·상태 확인(`ps`)·로그 확인(`logs`)·정리(`down`)

> 💬 **쉬운 풀이**: `hello-world` 는 "Docker가 잘 깔렸나" 확인하는 인사용 상자입니다. `ls` 는 서랍 안 목록 보기(list), `echo` 는 적은 말을 그대로 따라 말하기입니다. Compose의 `up` 은 켜기, `ps` 는 켜진 목록 보기, `logs` 는 일기 보기, `down` 은 끄고 치우기입니다.

필수 항목과 보너스 1번(Compose 기초)을 완료했고, 보너스 3번(Compose 운영 명령)도 같은 실습에서 함께 수행했습니다.

7. 어려웠던 점과 해결

문제가 생길 때마다 오류 메시지를 먼저 읽고, 원인을 추측한 뒤 확인하고 해결했습니다. README에는 5건을 기록했습니다.

무슨 일이 있었나	왜 그랬나 (추측 포함)	어떻게 해결했나
WSL 설치가 2%에서 멈춤 (Error 0x80d03805)	Microsoft Store를 거친 다운로드 실패	Store를 거치지 않는 `wsl --install --web-download` 로 재시도
`docker` 명령이 "permission denied"	Docker 연결 설정을 켜기 전에 열어 둔 터미널이라 새 설정이 반영 안 됨	`wsl --shutdown` 후 Ubuntu를 다시 열어 해결
내 폴더인데 들어갈 수 없음	`chmod 700` 을 `chmod 70` 으로 잘못 입력 → 주인 권한이 0	`chmod 700` 으로 복구
VSCode "게시 Branch" 버튼이 진행 안 됨	로그인 후 VSCode로 돌아오는 과정이 끊긴 것으로 추정	GitHub 웹에서 저장소를 만들고 터미널에서 직접 연결·push
모르는 파일 `README.md:Zone.Identifier` 가 생김	Windows가 인터넷에서 받은 파일에 붙이는 출처 꼬리표가 따로 파일로 생김	`rm` 으로 삭제 후 올림

> 💬 **쉬운 풀이:** `0x80d03805`는 병원 진단 코드처럼 오류 종류를 알려 주는 번호입니다. `wsl --shutdown`은 Windows 안의 리눅스를 완전히 껐다가 다시 켜는 명령이고, `rm`은 remove(지우기)의 줄임말입니다. "게시 Branch"는 VSCode에서 저장소를 GitHub에 처음 올리는 버튼입니다.

공통점은 오류 메시지에 원인의 단서가 있었다는 것입니다. 예를 들어 `Permission denied`는 말 그대로 "권한이 없다"는 뜻이었습니다.

8. 동료평가 예상 질문과 답

평가자가 물어볼 만한 질문을 미리 정리했습니다. 답은 한두 문장으로 말할 수 있게 연습합니다.

질문	짧은 답
Docker를 왜 쓰나요?	"내 컴퓨터에서만 돼요" 문제를 없애려고 씁니다. 프로그램과 필요한 것을 한 상자에 담아서 어디서나 똑같이 실행됩니다.
이미지와 컨테이너는 다른 건가요?	네. 이미지는 붕어빵 틀, 컨테이너는 그 틀로 만든 붕어빵입니다. 틀 하나로 여러 개를 만들 수 있습니다.
`-p 8080:80`에서 숫자 두 개는 뭐예요?	앞은 내 컴퓨터 문 번호, 뒤는 컨테이너 문 번호입니다. 8080으로 들어온 손님을 80번으로 안내합니다.
볼륨이 없으면 어떻게 돼요?	컨테이너를 지우면 안에 저장한 데이터도 함께 사라집니다. 볼륨에 저장하면 남습니다.
755와 644는 무슨 뜻이에요?	읽기 4, 쓰기 2, 실행 1을 더한 숫자를 나/그룹/나머지 순서로 적은 것입니다. 644는 나만 고칠 수 있고 다른 사람은 읽기만 합니다.
Git과 GitHub는 같은 건가요?	아닙니다. Git은 내 PC의 기록 도구, GitHub는 그 기록을 올려 공유하는 웹사이트입니다.
OrbStack을 안 썼는데 괜찮아요?	네. OrbStack은 서울캠퍼스의 sudo(관리자 권한) 제한 때문에 추가된 안내입니다. 경남캠퍼스인 저는 공통 요구사항대로 WSL2 + Docker Desktop을 설치했고, `docker` 명령은 같습니다. README 첫머리에도 이 내용을 적었습니다.
민감 정보는 어떻게 관리했나요?	비밀번호·토큰은 기록하지 않았고, Git 이메일은 GitHub가 주는 가림용 주소(noreply)를 썼습니다.

> 💬 **쉬운 풀이**: 토큰은 비밀번호 대신 쓰는 출입증 같은 문자열이라 남에게 보이면 안 됩니다. noreply 이메일은 진짜 이메일 주소를 가려 주는 대신 쓰는 주소입니다.

9. 배운 점·느낀 점

가장 어려웠던 순간

처음 환경을 설치하는 단계였습니다. WSL 설치가 오류로 멈추고, Ubuntu 비밀번호는 입력해도 화면에 보이지 않아 당황했습니다. 클로드에게 물어보고 나서야, 리눅스는 보안을 위해 비밀번호를 일부러 화면에 보여주지 않는다는 것을 알게 됐습니다. 오류 메시지를 하나씩 읽고 다른 방법(`--web-download`)으로 다시 시도해서 넘어갈 수 있었습니다.

"아, 이거구나!" 했던 개념

attach와 exec의 차이입니다. 설명만 들었을 때는 차이를 잘 몰랐는데, 직접 해보니 exec로 나오면 컨테이너가 살아 있고 attach로 나오면 꺼져 있었습니다. 바인드 마운트도 파일 하나를 고쳤을 때 8082만 바뀌고 8080은 그대로인 것을 보고 나서야 "복사본"과 "실시간 연결"의 차이가 이해됐습니다.

비전공자로서 처음 써본 느낌

명령어를 칠 때 오타가 많았습니다. 띄어쓰기 하나, 점 하나(`git add .`)만 빠져도 결과가 달라졌습니다. 그래서 컴퓨터는 적은 그대로만 움직인다는 것을 실감했습니다. `chmod 70` 오타는 오히려 폴더 권한을 제대로 이해하는 계기가 됐습니다.

다시 한다면 다르게 하고 싶은 점

실습하면서 바로바로 스크린샷과 로그를 정리하고 싶습니다. 이번에는 스크린샷을 중간에야 챙기기 시작했고, README를 마지막에 한꺼번에 정리했습니다. 다음에는 단계마다 기록하면서 진행하고 싶습니다.